Determination of 3D Surface Structure using Grid Laser and Image Processing

Report submitted to
Indian Institute of Technology, Kharagpur
For the fulfilment of MTP-II in
Mechanical Engineering with specialization in
"Manufacturing Science and Engineering"

Submitted by
Suman Das
21ME31041

Under the Supervision of
Prof. Vikranth Racherla
and
Prof. Anirvan Dasgupta

Department of Mechanical Engineering
Indian Institute of Technology, Kharagpur – 721302
April, 2026

CERTIFICATE

It is certified that the work contained in the thesis titled **"Determination of 3D Surface Structure using Grid Laser and Image Processing"**, by **" Mr Suman Das"**, has been carried out under our supervision and that this work has not been submitted elsewhere for a degree.

Prof. Vikranth Racherla
Department of Mechanical Engineering
Indian Institute of Technology, Kharagpur
April, 2026

ADasGupta
Prof. Anirvan Dasgupta
Department of Mechanical Engineering
Indian Institute of Technology, Kharagpur
April, 2026

Abstract

This project presents a hybrid 3D reconstruction system based on stereo vision and laser grid projection, designed to generate accurate 3D point clouds without the use of deep learning or high-cost sensors. A stereo camera is used to capture images of a scene illuminated by a structured laser grid. By detecting and matching laser points across the stereo pair, depth is computed via triangulation and later converted into a full 3D point cloud. To validate and tune the method, the entire setup was also simulated in Blender using a virtual stereo camera and laser projection, allowing controlled testing of geometry, calibration, and lighting effects. The system was implemented, calibrated, and evaluated on both real and simulated data. Results indicate that the laser-assisted stereo approach improves correspondence accuracy compared to passive stereo, while remaining significantly cheaper than commercial structured-light scanners or LiDAR devices. The project concludes with discussion of accuracy, limitations, and potential extensions such as denser patterns or real-time scanning.

TABLE OF CONTENTS

Introduction

Stereo vision is a widely used technique for depth estimation, but passive stereo suffers from correspondence ambiguity, especially on texture less or uniform surfaces. To overcome this, active stereo systems introduce projected patterns — typically structured light — to improve feature matching. While commercial systems use high-resolution projectors, this project explores a simpler and low-cost alternative: a **laser grid projector**, which generates sparse but highly detectable feature points.

The motivation for this work is to design a complete workflow — from hardware setup to point cloud output — that can be reproduced, tested, and compared both in the real world and in simulation. Blender was used to create a photorealistic virtual stereo setup to test geometry, camera calibration, and point detection without physical constraints. The combination of **stereo vision + active grid projection** aims to produce an accurate 3D reconstruction pipeline with minimal hardware and no deep learning.

Objectives

The main objectives of the project are:

- Design and build a stereo + laser grid-based 3D scanning system
- Implement detection, correspondence, triangulation, and point cloud generation
- Develop a simulation equivalent in Blender for controlled calibration and testing
- Export final results as PLY / OBJ point clouds for visualization in Blender.
- Document the full pipeline so it can be reproduced or extended by others

Literature Review

3D reconstruction from images has traditionally been approached using **passive stereo vision**, where depth is obtained by matching corresponding pixels across two or more cameras. Classical stereo algorithms such as block matching, semi-global matching (SGM), and feature-based correspondence work well on textured scenes, but fail on **low-texture, reflective, or repetitive surfaces**. This limitation has motivated the use of **active illumination**, where an external light pattern is introduced to simplify correspondence.

Passive Stereo Vision

Traditional stereo pipelines rely purely on appearance-based matching and therefore require strong visual texture. Scharstein and Szeliski (2002) established the standard taxonomy of stereo matching algorithms, and later work improved accuracy using cost aggregation, graph cuts, and variational methods. However, these methods suffer from ambiguity in poorly textured scenes, specular objects, or scenes with low contrast. Deep learning-based stereo networks such as PSMNet (2018) and RAFT-Stereo (2021) significantly improve performance but require large training datasets and powerful computing hardware, making them unsuitable for low-cost embedded systems. **Motivation:** passive stereo alone is insufficient for the kind of scenes targeted in this work.

Structured-Light and Laser-Based 3D Scanning

Structured-light systems project a known pattern (e.g., grid, dots, stripes) and observe its deformation on the surface. The most common industrial implementation is the **laser line scanner**, where a 2D camera and a laser plane form a triangulation geometry. Geng (2011) provides a comprehensive review of structured-light approaches, including single-line, multi-line, binary patterns, and phase-shifting techniques. These systems provide reliable depth even on texture less surfaces. Usamentiaga et al. (2014) introduced a two-stripe laser scanner to compensate for vibrations in industrial inspection, demonstrating how multiple projected lines can increase robustness. He et al. (2017) proposed an accurate centreline extraction method for laser stripes using a Hessian-based approach to improve subpixel localization. More recent works (Wang, 2021; MDPI, 2020-2024) have focused on **low-cost laser scanning**, showing that commodity cameras and inexpensive lasers can produce accurate point clouds with proper calibration.

Relevance: These systems prove that active laser patterns solve correspondence failure—but most use a single scan line, not a full grid

Multi-Line and Grid-Based Projection

While single laser stripes reconstruct only one profile at a time, recent research has explored **multi-line projection** and even **full laser grids** to obtain denser point clouds without mechanical scanning. A 2024 study by Nature Scientific Reports demonstrated multi-line projection with quadric surface modelling to increase reconstruction density. Other works (2025, Optica) show that active laser patterns are especially valuable for **weak-texture 3D reconstruction**, directly aligning with the motivation for adding a laser grid to stereo. However, most multi-line systems are either dependent on special decoding algorithms, or use expensive structured-light projectors.

Gap: Very few works combine **stereo + static laser grid** for 3D reconstruction using off-the-shelf hardware, which is the niche addressed by the present work.

Experiment 1

Experimental Methodology

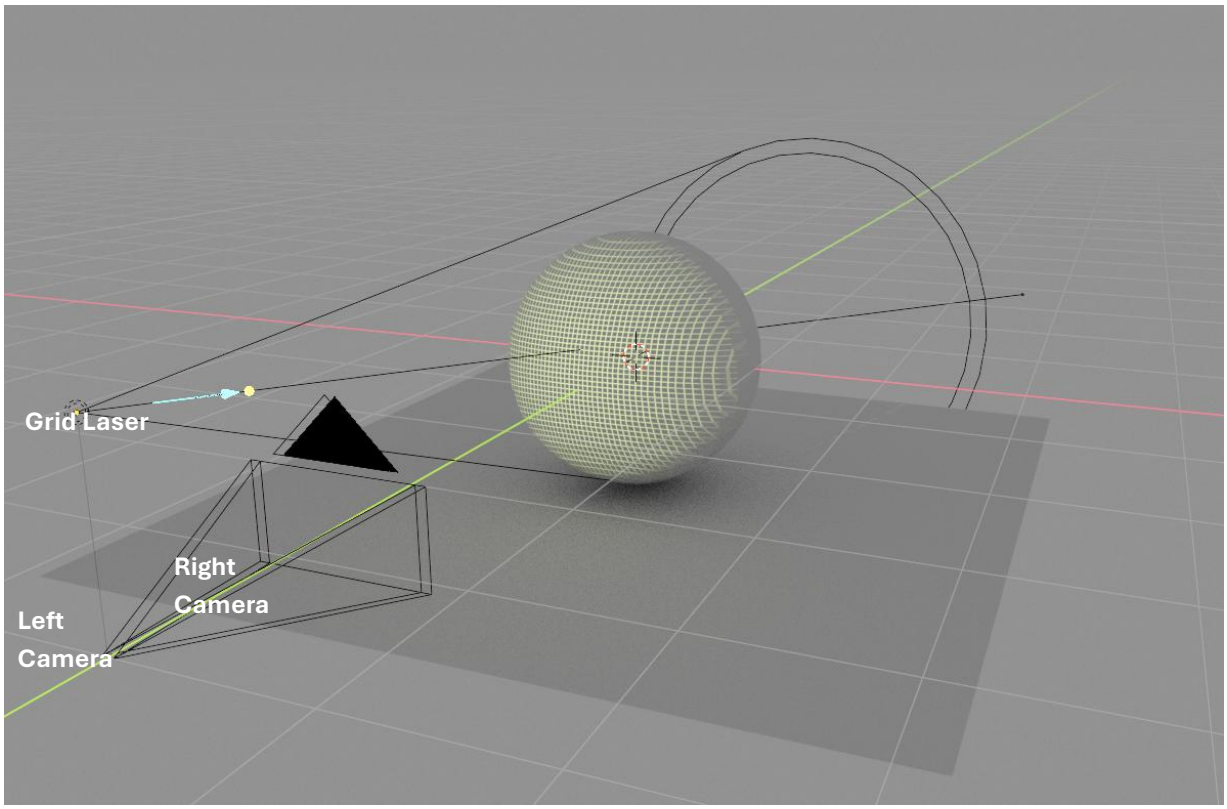


Fig1: Experimental Setup

Stereo camera : Left and right camera at a distance of 2 Cm looking parallel.
Specifications of the camera sensors are,

Focal length : 50mm
Sensor Height : 36mm
Sensor Weight : 24mm
Resolution : 1920x1080

Grid Laser : Laser is used to project the grid on an object
Point Lasar (blue) : Used to detect a known corresponding point.
Sphere : Used as an object to measure.

Problem Statement

Constructing the 3d representation of the grid projected on the object.

Methodology

We started by capturing 2 stereo images of a red laser grid on a spherical surface. in Blender[fig2, 3]. There is also a blue dot used to find a corresponding point. First step is to isolate the blue point by thresholding using HSV range. The image is converted into grayscale and threshholded and converted into a white grid [fig3]. Then we apply box blur and normalized (converting grayscale values to its full range) the image. Using blue dots then match left to right intersection using nearest-neighbour[fig6, 7].

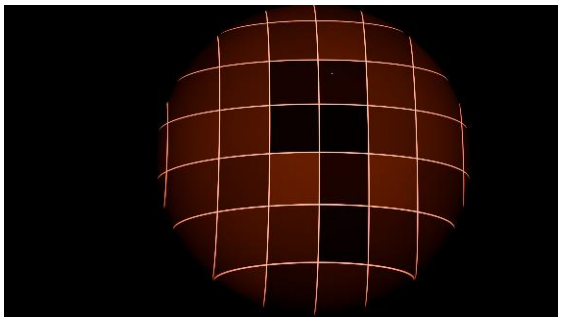


Fig2: Left Image from stereo camera

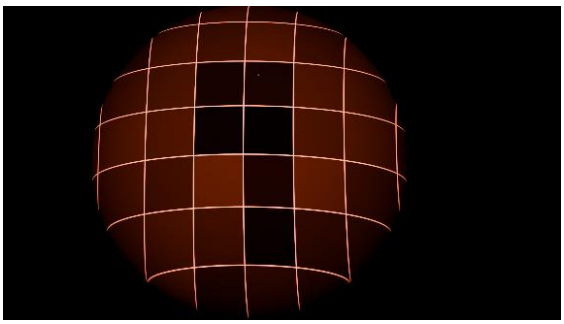


Fig3:Right Image from stereo camera

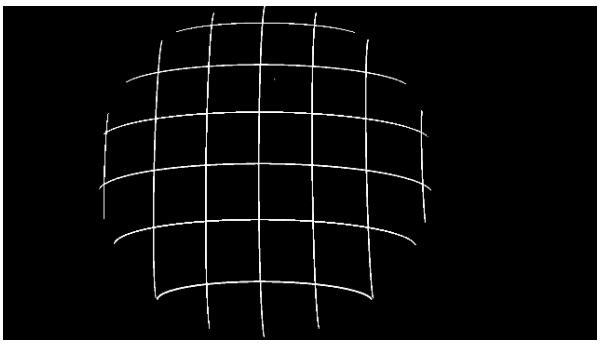


Fig4: White Grid of the Right Image

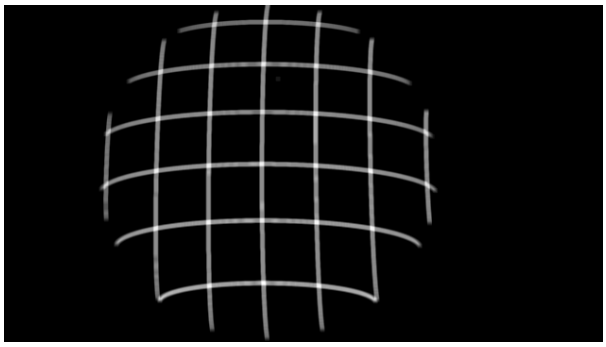


Fig5: Blurred and Normalized Grid

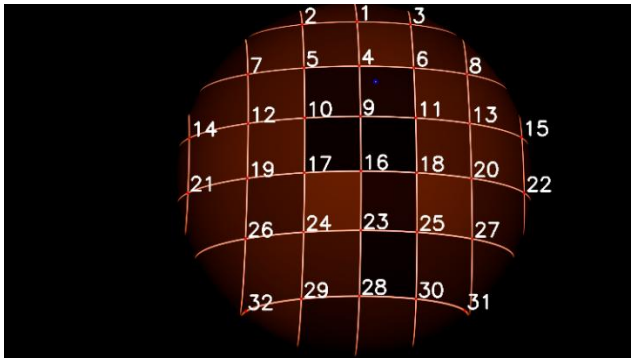


Fig6 : Left Image with intersection numbered

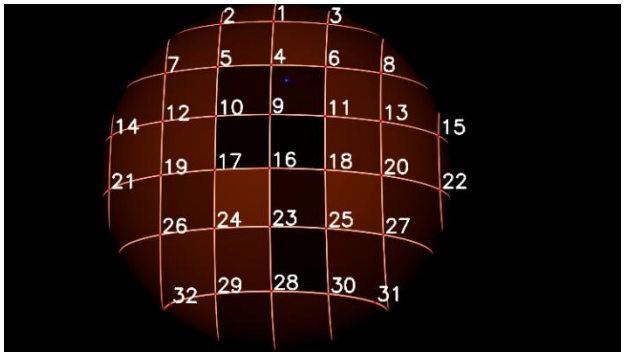
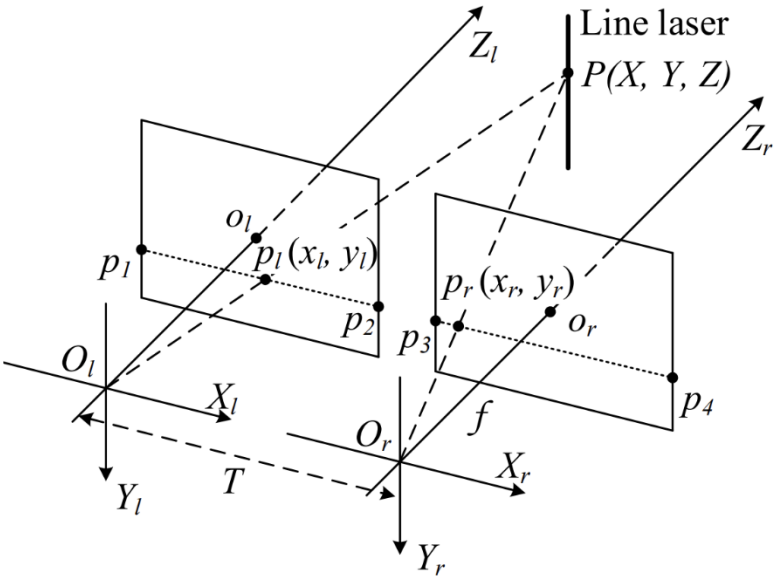


Fig7 : Right Image with intersection numbered

Furthermore, we assume that the camera focus is f, the disparity is d ($d = x_l - x_r$), the baseline is T. Thus, the 3D coordinates of point P on the laser stripe can be obtained with Eq. (1) [fig 8]. After detecting 3D points we move on to detect the lines connecting the points by A* algorithm [fig 9]. Although the algorithm generates a path contained in the extracted laser line it doesn't follow the leaser line centre. To solve this, we first create a region covering the A* detected line [orange line in fig 9] and then use thinning in python to get the centreline more accurately [fig 10].



$$\begin{cases} X = \dfrac{x_l * T}{d} \\ Y = \dfrac{y_l * T}{d} \\ Z = \dfrac{f * T}{d} \end{cases} \quad (1)$$

Fig8 : Stereo Camera Arrangement [1]

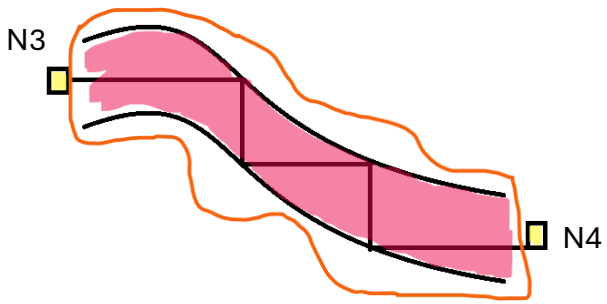


Fig9 : A* Path Representation between two intersection

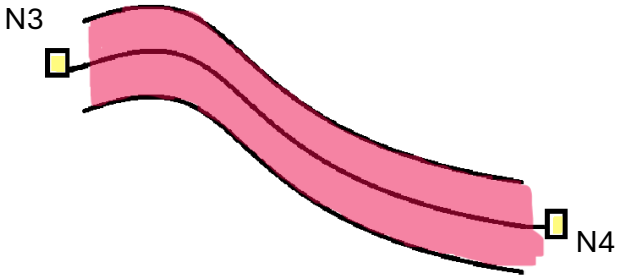


Fig10 : Smooth Path / Centreline between two intersection

We then move on to detect neighbours.

Step 1 : Detect the neighbours. Simply if two nodes are connected a line they are neighbours. We save this data.

Step 2 : Detect all N_4 nodes (node with 4 neighbours).

Step 3 : From the data saved we find the neighbours of the N_4 nodes.

Step 4 : Classify the neighbours as horizontal or vertical for the N_4s. This is done by comparing their position compared to x (horizontal) and y (vertical) axis. Two closest to the horizontal are horizontal nodes and the other two are vertical nodes.

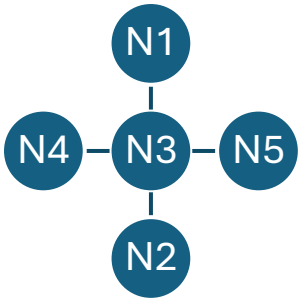


Fig11 : N_4 Node

Now we can form the horizontal and vertical disks. Let say we want to make a horizontal disk. To do this we start with a N_4 node and look for its horizontal neighbours and then look for horizontal neighbours of the horizontal neighbours. We do this until all the N4 nodes are iterated. After we iterate them, we now have disks (collection of nodes in a laser plane). From the example above [N6, N4, N3, N5, N8, N10] is disk.

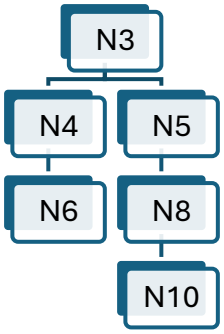


Fig12 : Disk Representation

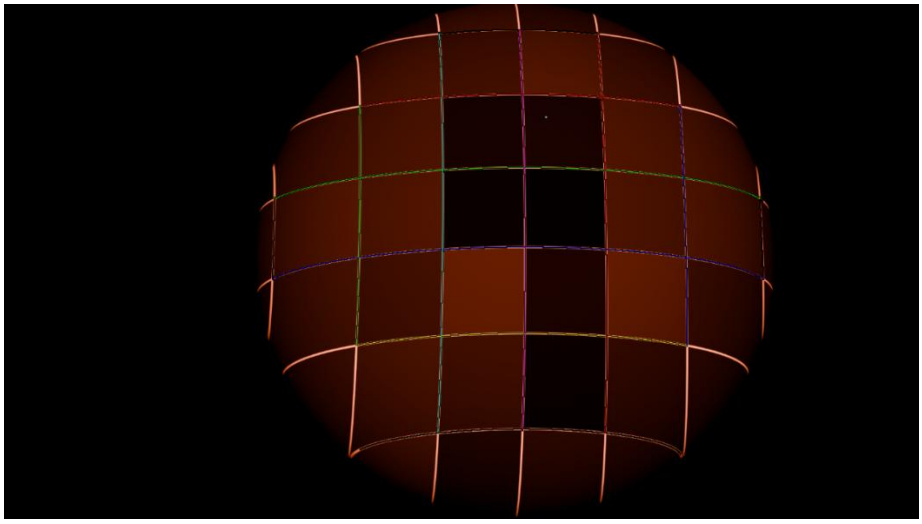


Fig13 : Horizontal and Vertical Disks Shown by Different Colours

We the correct the tilt of the points in the horizontal disks due to the angle between camera plane and laser plane by multiplying with the below matrix (2) [fig 14(a)].

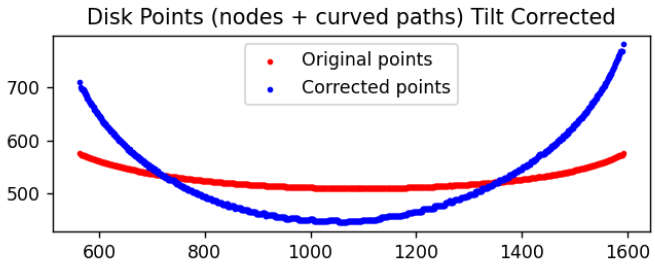
$$z = y^* \tan(\theta)$$

$$new\ points = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix}^T . points\ extracted \qquad (2)$$

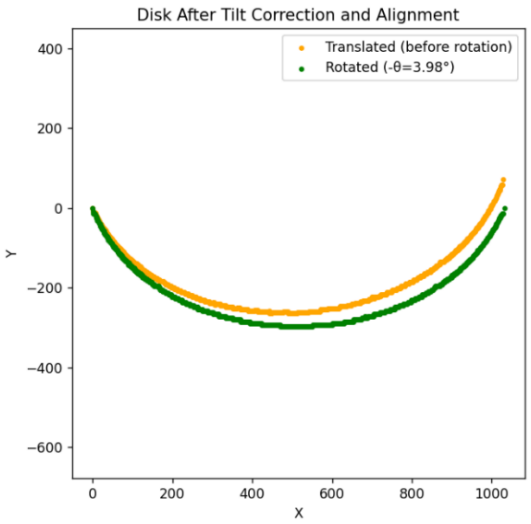
After this we plot and transform the detected points to make the start and end points lie on the x axis [fig 14(b)].

If we have a set of points in 2D we can have 2 arrangements. To find the correct orientation we look for the intermediate nodes in 2D and in 3D to see their arrangement along the vector from one end point to the other end point by taking cross product of $\overrightarrow{AB}$ and $\overrightarrow{AP_x}$ for 3D points and corresponding 2D points then taking cross product with normal vector of the plane formed by the nodes in the disk. We can check for the sign by the Eq. (3) to identify the correct arrangement. P_x are intermediate node points of a disk [fig15].

$$Sign = sign(\overrightarrow{normal} \cdot (\overrightarrow{AB} \times \overrightarrow{AP})) \qquad (3)$$



(a)



(b)

Fig14 : Curve Transformations

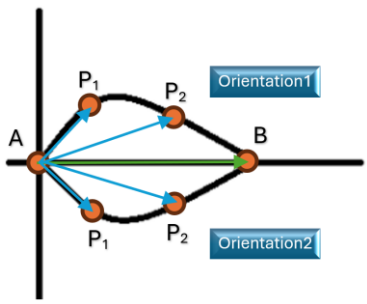
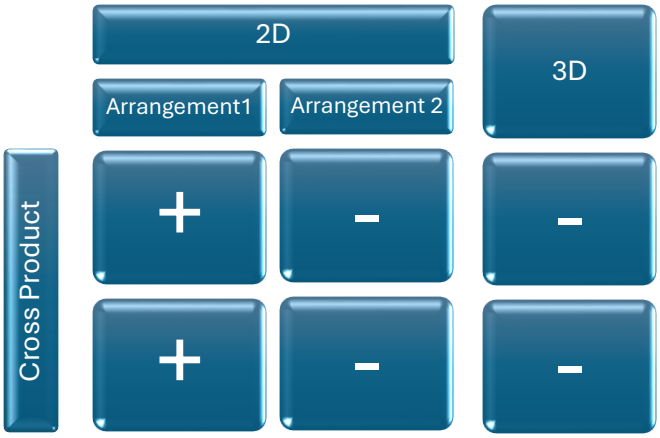


Fig15 : Dual Representation

As an example,



Here the second arrangement is correct.

After having the correct arrangement, we can move on to converting the 2D points to 3D. To do that we find the x coordinate of the points. The projection of the point devides the line joining the 1st (0, 0) and last (X, 0) node in the ratio of L1 and L2. So we also devide the points in the 3D by the same ratio. The new point we get let say N. We then find the cross product of plane normal and $\overrightarrow{AB}$. and the 3D position of the point (X1, Y1) is given by Eq. (4)

$$Pos = N + m.\frac{Y1.(\overrightarrow{normal} \times \overrightarrow{AB})}{\|\overrightarrow{normal} \times \overrightarrow{AB}\|} \quad (4)$$

$$N = \frac{L1 * Q2 + L2 * Q1}{L1 + L2} \quad (Q1 \text{ and } Q2 \text{ are end nodes})$$

$$m = \frac{\|Q2 - Q1\|}{|X|}$$

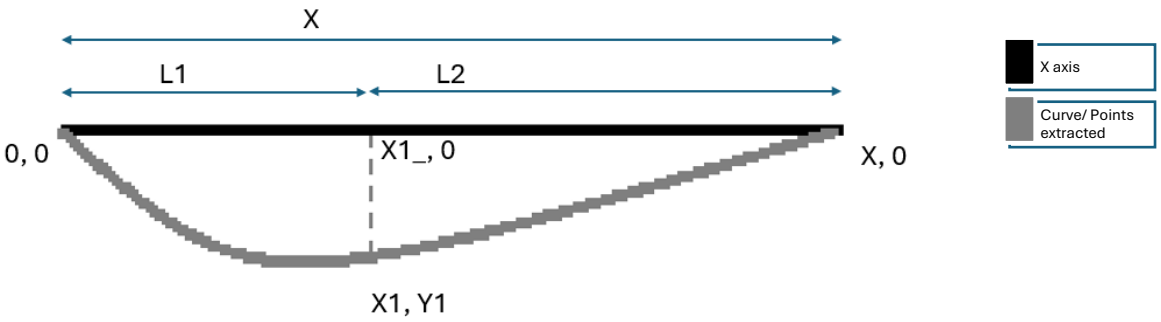


Fig16 : Extracted Points of a disk

Results and discussion

This project demonstrates that a low-cost 3D reconstruction system can be built using a stereo camera and a projected laser grid, without relying on deep learning or expensive structured-light hardware. By detecting the intersection points of the laser grid across the stereo image pair, triangulating them, and converting the results into a point cloud, it is possible to obtain accurate 3D measurements even in scenes with low texture where passive stereo would fail. Heres an representation of point cloud of the nodes in 3d and the final 3D disks(horizontal).

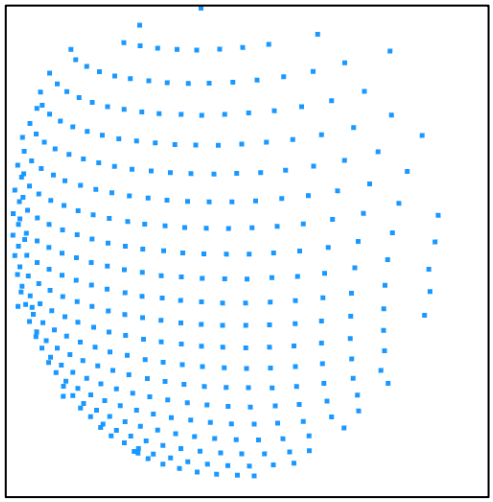


Fig17 : 3D nodes(of dense intersection)

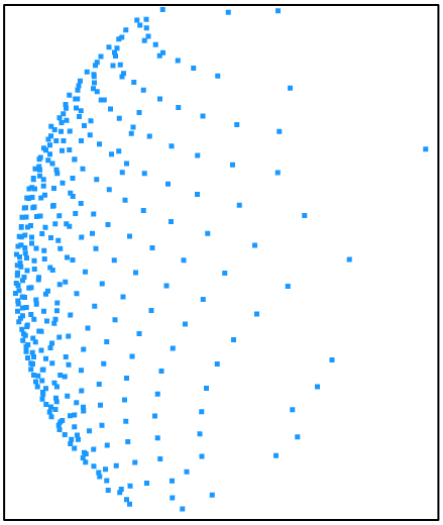


Fig18 : 3D nodes (of dense intersection)

Fig19 : 3D disks

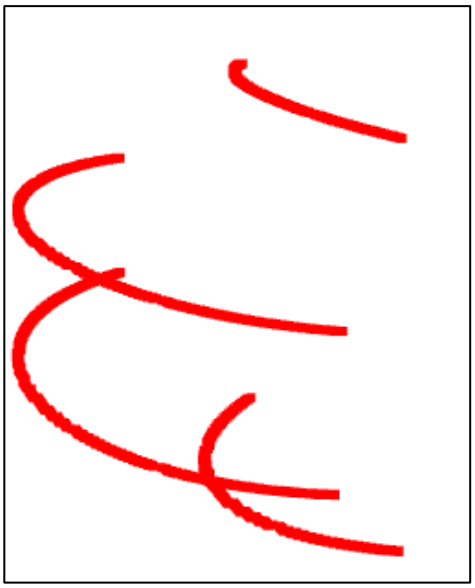


Fig20 : 3D disks

Experiment 2

Experimental Methodology

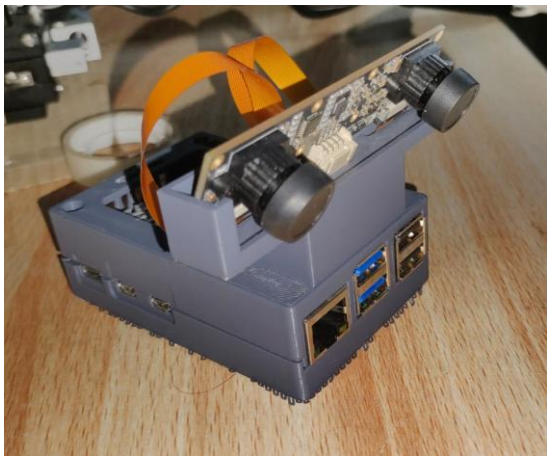


Fig 21 :raspberry pi 5 and stereo camera in a casing

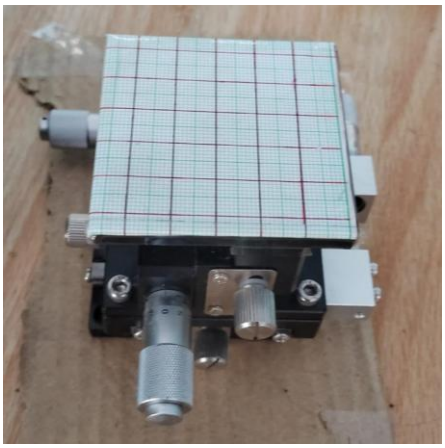


Fig 22 : XYZ table

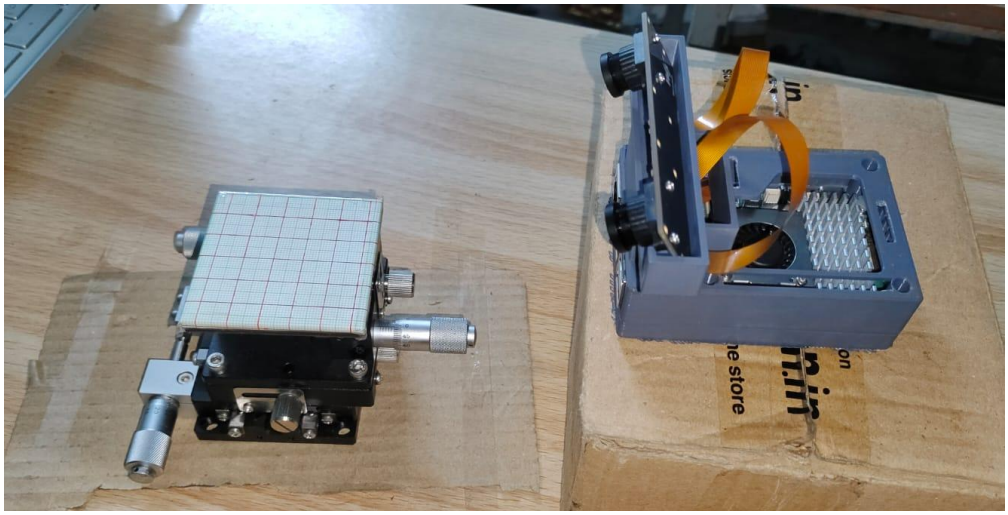


Fig 23 : setup with pi5 camera and XYZ table

Problem Statement

Constructing the 3d representation of the grid on a XYZ table.

Methodology

A grid was drawn on paper and pictures were taken with the stereo camera using raspberry pi5 every time the XYZ table was moved up by 0.25 mm upwards. Then a section with a square with four corners properly visible is extracted from all the images i.e. left images and right images. Then we use masking, gaussian blurring and thresholding to get the four corners or line intersections using same methods as in experiment1. After this step the 2d coordinates in the sensor (in pixels) that we get from extracting intersections in the previous step was used to find 3D co-ordinate in the left camera axis using formulas below.

$$\begin{cases} X = \dfrac{x_l * T}{d} \\ Y = \dfrac{y_l * T}{d} \\ Z = \dfrac{f * T}{d} \end{cases}$$

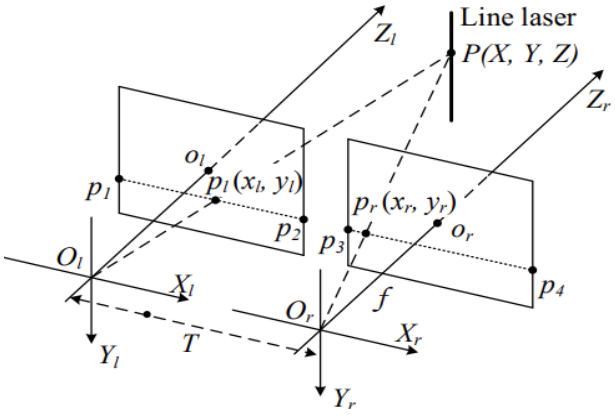


Fig 24 : Stereo Camera Arrangement

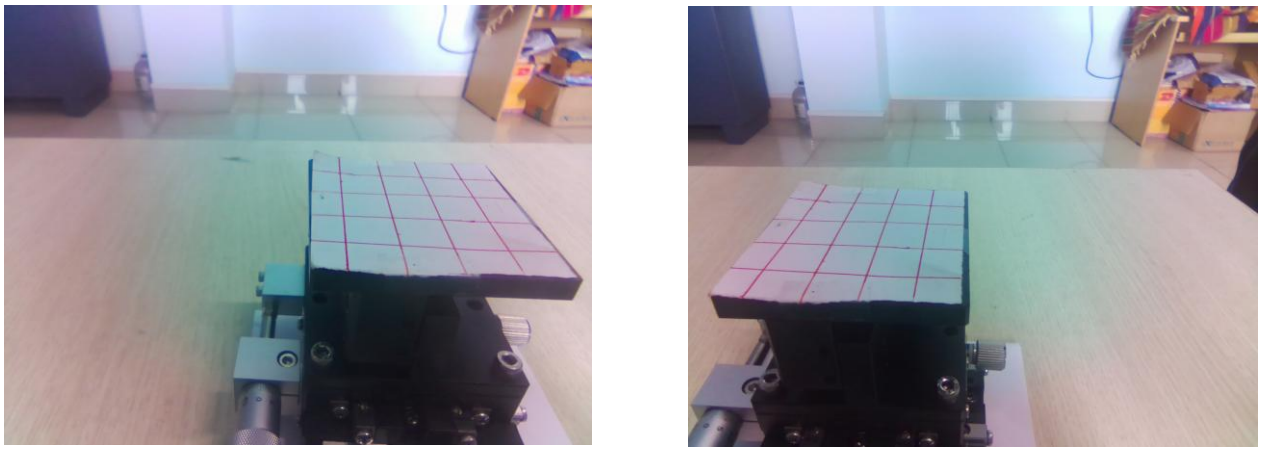


Fig 25 : Stereo Images captured from left and eight cameras repectively

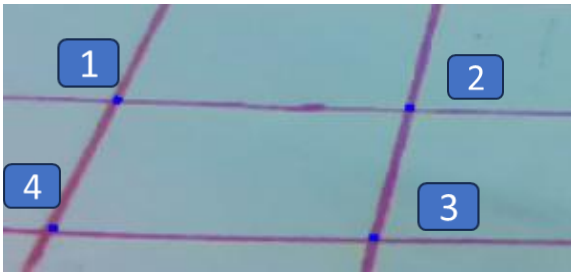


Fig 26 : Corners identified for a square by python (open cv) (shown in blue)

These coordinates were plotted to see if they form perfect squares as they are in the actual grid. The deviations from a square were also calculated for all the squares. Then the change in distances were calculated between images for all four points and plotted.

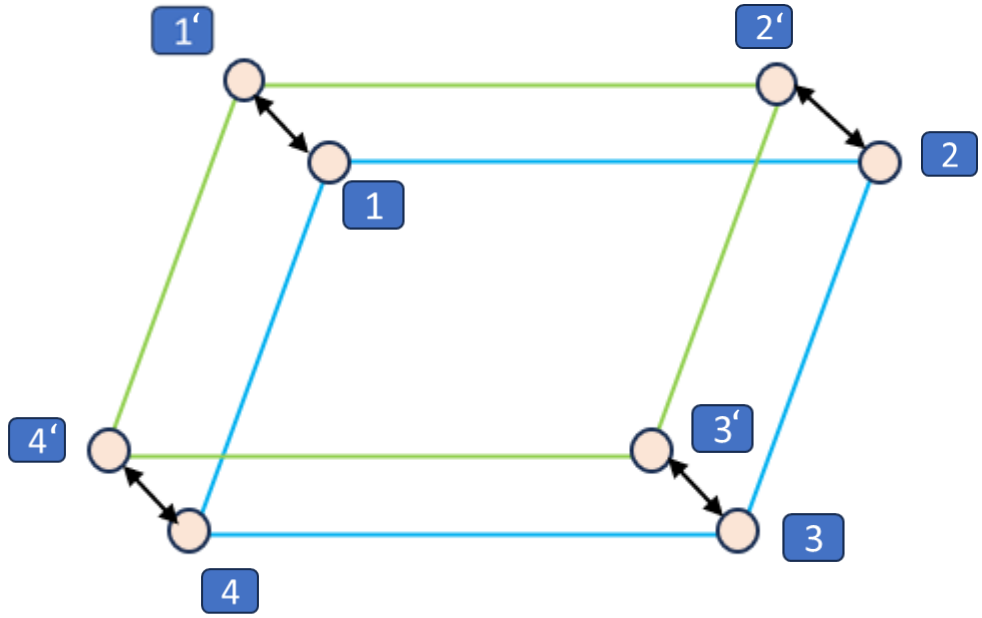


Fig 27 : Visualization of the square moving in space due to change of height of the XYZ-table as seen by Right camera. Where blue square is the initial position (1-2-3-4) and green is the second position (1’-2’-3’-4) after change of height of XYZ table by 0.25mm. The double-sided black arrows shows the change in position by all the four corners. See fig 28 for same experimental result.

Results and discussion

The distances for point1 are measured from image2 to image1, image3 to image2 and so on. By doing this the delta data of point1 was got. Then it was done for all the other 3 points of the square. The exact value of all the deltas should be 0.00025m or 0.25 mm. The data was between 0.4mm to 0.1 mm.

After this the points from all the images were plotted using the 3D data to create parallel squares in 3D space. the error or deviations from the actual side measurement and from ideal square structure was also calculated. The average was around 5 % error. We used this formula to get the error.

Error = (percent error in side length + percent error in diagonal)/2

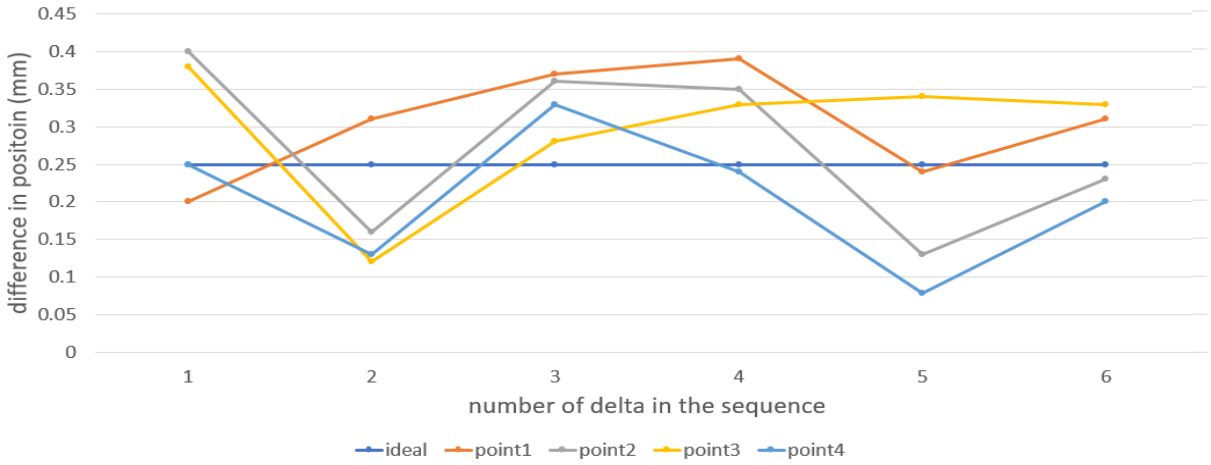


Fig 28: change in positions in consecutive images for four points

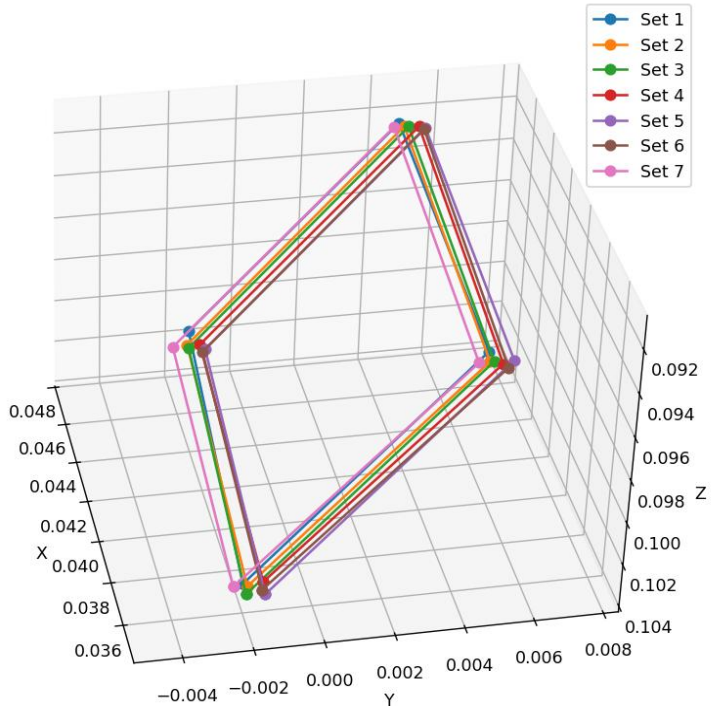


Fig 29: All the squares plotted in 3d dimensions are in m

Conclusions

The point cloud generation was validated in a controlled Blender simulation, showing that simulated environments can serve as a useful testbed for calibration, algorithm development, and error analysis before physical implementation. An additional contribution of this work is the extraction and visualization of horizontal grid lines in 3D space, which can be used not only for visualization but also for evaluating geometric distortions, surface curvature, and calibration accuracy. Several key lessons emerged from this work:

1. **Active stereo significantly improves correspondence reliability** — Introducing a laser grid eliminates many of the matching ambiguities found in pure passive stereo, especially on smooth or uniform surfaces.
2. **Geometric calibration is more important than dense data** — Even a sparse but accurately triangulated grid can produce meaningful 3D reconstruction, demonstrating that precision in calibration can compensate for lower point density.
3. **Simulation is a valuable part of the development pipeline** — Blender provided a repeatable and noise-free environment for testing camera parameters, pattern geometry, and triangulation formulas before building the physical setup, reducing trial-and-error effort.
4. **Point cloud usability depends on post-processing structure** — Extracting and plotting the 3D laser grid connections (horizontal nodes and connecting lines) helps in interpreting the scan, transforming raw points into a more structured representation.
5. **Low-cost scanning systems are feasible with open tools** — The entire pipeline was built using open-source software (Blender, Python, OpenCV), standard cameras, and a simple laser grid, showing that 3D reconstruction does not require proprietary hardware.

Future Scope

- Extend the system from single-frame capture to **video-based scanning**, enabling continuous 3D reconstruction while the laser or object moves.
- Improve point cloud density by replacing the fixed grid with a **sweeping laser line or dynamic pattern**.
- Integrate **real-time processing.**
- Apply surface fitting or meshing methods to convert the sparse grid into a full 3D surface model.

References

[1] Jian, X., Chen, X., He, W., & Gong, X. (2020). Outdoor 3D reconstruction method based on multi-line laser and binocular vision. IFAC-PapersOnLine, 53(2), 9554-9559.

[2] Hu, Y., Ma, Y., Xu, M., & Chen, X. (2016, June). Measurement algorithm of mounting holes based on binocular stereo vision. In *2016 IEEE International Conference on Cyber Technology in Automation, Control, and Intelligent Systems (CYBER)* (pp. 489-492). IEEE.

[3] Geng, J. (2011). Structured-light 3D surface imaging: A tutorial. *Optical Engineering*, 3(2), 128.

[4] Usamentiaga, R., Molleda, J., & Garcia, D. F. (2014). Structured-Light Sensor Using Two Laser Stripes for 3D Reconstruction without Vibrations. *Sensors*, 14(11), 20041-20063.

[5] He, L., Wu, S., & Wu, C. (2017). Robust laser stripe extraction for three-dimensional reconstruction based on a cross-structured light sensor. *Applied Optics*, 56(4), 823-832.

[6] Xu, G., Chen, F., Chen, R., Li, X., & ... (2020). Vision-based reconstruction of laser projection with invariant composed of points and circle on 2D reference. *Scientific Reports*, 10, Article 11866.

[7] Zhao, C., Yang, J., Zhou, F., Sun, J., Li, X., & Xie, W. (2020). A robust laser stripe extraction method for structured-light vision sensing. *Sensors*, 20(16), 4544.

[8] MTP - Kanhu Hansdah (21ME63R01)

Acknowledgement

I would like to express my sincere gratitude to my supervisors, Prof. Vikranth Racherla and Prof. Anirban Dasgupta, for their continuous guidance, encouragement, and valuable insights throughout the course of this project. Their expertise, constructive feedback, and support were instrumental in shaping both the technical development and the academic quality of this work.

Finally, I extend my appreciation to my friends and family for their patience, motivation, and support during the completion of this work.

Appendix

Python code to extract point and convert to 3D

```
import cv2
import numpy as np

import os

m_max = 5
y_values = [0, 5]
c_values = [0, 25]

with open("points_3D.txt", "w") as f:

    for m in range(m_max):
        for y in y_values:
            for c in c_values:

                nameL = f"camL_m{m}_{y}_c{c}.jpg"
                nameR = f"camR_m{m}_{y}_c{c}.jpg"

                if os.path.exists(nameL) and os.path.exists(nameR):
                    # Read Left image
                    img = cv2.imread(nameL)

                    # Get height and width
                    h, w, _ = img.shape

                    # Black out unwanted regions
                    img[0:int(1.78*h/4), :] = 0
                    img[int(1.76*h/3):int(h), :] = 0
                    img[:, 0:int(2.9*w/4)] = 0
                    img[:, int(6.96*w/8):-1] = 0

                    # Convert to HSV
                    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

                    # Define red ranges
                    lower_red1 = np.array([130, 50, 70])
                    upper_red1 = np.array([170, 255, 255])

                    # Create mask
                    mask = cv2.inRange(hsv, lower_red1, upper_red1)

                    # Apply Gaussian Blur to smooth edges
                    blurred_mask = cv2.GaussianBlur(mask, (91, 91), 0)

                    # Threshold the blurred mask
                    _, thresh = cv2.threshold(blurred_mask, 80, 255, cv2.THRESH_BINARY)
```

```
            # Find connected components
            num_labels, labels, stats, centroids = v2.connectedComponentsWithStats(thresh)

            points_left = []  # create list

            for i in range(1, num_labels):  # skip background (0)
               # Create mask for this component
               component_mask = (labels == i).astype("uint8") * 255

               # Apply mask to blurred image
                              masked_blur = cv2.bitwise_and(blurred_mask, blurred_mask,
mask=component_mask)

               # Find max intensity and its location
               minVal, maxVal, minLoc, maxLoc = cv2.minMaxLoc(masked_blur)

               points_left.append(maxLoc)  # add point to list

               print(f"Component {i}: Max Value = {maxVal}, Location = {maxLoc}")

            # Read Right image
            img = cv2.imread(nameR)

            # Get height and width
            h, w, _ = img.shape

            # Black out unwanted regions
            img[0:int(1.97*h/4), :] = 0

            img[int(1.9*h/3):int(h), :] = 0
            img[:, 0:int(1.2*w/4)] = 0
            img[:, int(3.67*w/8):-1] = 0

            # Convert to HSV
            hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

            # Define red ranges
            lower_red1 = np.array([110, 30, 30])
            upper_red1 = np.array([170, 255, 255])

            # Create mask
            mask = cv2.inRange(hsv, lower_red1, upper_red1)

            # Apply Gaussian Blur to smooth edges
            blurred_mask = cv2.GaussianBlur(mask, (91, 91), 0)

            # Threshold the blurred mask
            _, thresh = cv2.threshold(blurred_mask, 90, 255, cv2.THRESH_BINARY)

            # Find connected components
```

```
num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(thresh)

points_right = []

for i in range(1, num_labels):  # skip background (0)
    # Create mask for this component
    component_mask = (labels == i).astype("uint8") * 255

    # Apply mask to blurred image
                masked_blur = cv2.bitwise_and(blurred_mask, blurred_mask,
mask=component_mask)

    # Find max intensity and its location
    minVal, maxVal, minLoc, maxLoc = cv2.minMaxLoc(masked_blur)

    points_right.append(maxLoc)

    print(f"Component {i}: Max Value = {maxVal}, Location = {maxLoc}")

# Camera parameters
f_mm = 2.6
sensor_width_mm = 3.68
sensor_height_mm = 2.76

# Image size
h, w = img.shape[:2]

# Convert focal length to pixels
f_x = (f_mm / sensor_width_mm) * w
f_y = (f_mm / sensor_height_mm) * h

# Principal point (approx center)
c_x = w / 2
c_y = h / 2

# Baseline
B = 0.06  # meters

# Your detected points
# points_left = [(2434, 1199), (2705, 1213), (2511, 1361), (2815, 1373)]
# points_right = [(2400, 1198), (2670, 1212), (2475, 1360), (2780, 1372)]

# Compute 3D coordinates
points_3D = []

for (xL, yL), (xR, yR) in zip(points_left, points_right):
    d = xL - xR

    if d == 0:
        continue
```

```
                Z = (f_x * B) / d
                X = ((xL - c_x) * Z) / f_x
                Y = ((yL - c_y) * Z) / f_y

                points_3D.append((X, Y, Z))

                print(f"Pixel: ({xL},{yL}) -> 3D: X={X:.4f}, Y={Y:.4f}, Z={Z:.4f}")

            for (X, Y, Z) in points_3D:
                f.write(f"{X:.6f}, {Y:.6f}, {Z:.6f}\n")
            f.write("\n")
```

python code to check square

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import combinations

# ------------------------------
# Load data from file
# ------------------------------
def load_points(filename):
    sets = []
    current_set = []

    with open(filename, 'r') as f:
        for line in f:
            line = line.strip()

            if not line:
                if current_set:
                    sets.append(current_set)
                    current_set = []
                continue

            point = list(map(float, line.split(',')))
            current_set.append(point)

    if current_set:
        sets.append(current_set)

    return np.array(sets)  # shape: (num_sets, 4, 3)

# ------------------------------
```

```
# Pairwise distances (3D)
# ------------------------------
def pairwise_distances(points):
    return np.array([
        np.linalg.norm(p1 - p2)
        for p1, p2 in combinations(points, 2)
    ])

def square_score(points):
    dists = np.sort(pairwise_distances(points))

    sides = dists[:4]
    diagonals = dists[4:]

    # Mean values
    side_mean = np.mean(sides)
    diag_mean = np.mean(diagonals)

    # Errors (normalized)
    side_var_error = (0.0127 - side_mean) / side_mean
    diag_var_error = (0.0127*np.sqrt(2) - diag_mean) / diag_mean

    total_error = (side_var_error + diag_var_error)/2

    return 1-total_error

# ------------------------------
# Square check
# ------------------------------
def is_square(points, tol=1e-4):
    dists = np.sort(pairwise_distances(points))

    sides = dists[:4]
    diagonals = dists[4:]

    return (
        np.allclose(sides, sides[0], atol=tol) and
        np.allclose(diagonals, diagonals[0], atol=tol) and
        np.isclose(diagonals[0], np.sqrt(2)*sides[0], atol=tol)
    )

# ------------------------------
# Order points around centroid
```

```
# --------------------------------
def order_points(points):
    pts_2d = points[:, :2]  # project to XY

    centroid = np.mean(pts_2d, axis=0)

    angles = np.arctan2(
        pts_2d[:, 1] - centroid[1],
        pts_2d[:, 0] - centroid[0]
    )

    order = np.argsort(angles)
    return points[order]

# --------------------------------
# MAIN
# --------------------------------
data = load_points("points_3D.txt")

num_sets = len(data)
print(f"Total sets: {num_sets}\n")

# Check each set
for i, pts in enumerate(data):
    score = square_score(pts)
    print(f"Set {i+1}: Square score = {score:.4f}")

# --------------------------------
# Plot (XY projection)
# --------------------------------
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

for i, pts in enumerate(data):
    pts_ordered = order_points(pts)

    x = pts_ordered[:, 0]
    y = pts_ordered[:, 1]
    z = pts_ordered[:, 2]

    # close the loop
```

```
    x = np.append(x, x[0])
    y = np.append(y, y[0])
    z = np.append(z, z[0])

    ax.plot(x, y, z, marker='o', label=f'Set {i+1}')

ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
ax.set_title("Squares in 3D Space")

ax.legend()


def set_axes_equal(ax):
    x_limits = ax.get_xlim3d()
    y_limits = ax.get_ylim3d()
    z_limits = ax.get_zlim3d()

    x_range = abs(x_limits[1] - x_limits[0])
    y_range = abs(y_limits[1] - y_limits[0])
    z_range = abs(z_limits[1] - z_limits[0])

    max_range = max(x_range, y_range, z_range)

    x_mid = np.mean(x_limits)
    y_mid = np.mean(y_limits)
    z_mid = np.mean(z_limits)

    ax.set_xlim3d([x_mid - max_range/2, x_mid + max_range/2])
    ax.set_ylim3d([y_mid - max_range/2, y_mid + max_range/2])
    ax.set_zlim3d([z_mid - max_range/2, z_mid + max_range/2])

set_axes_equal(ax)

plt.show()
```